8 Approximation Algorithms

Approximation algorithms don't always return an optimal solution, but their solutions are often close to optimal. They always run in polynomial time, and they are often relatively simple. (Again, in this chapter, exact running times won't be essential for our purposes.)

Overview

Section	Summary	Approx. ratio
8.1: Vertex Cover	Return endpoints of any maximal matching	2
8.2: Load Balancing	Greedy: Assign each job to machine with smallest current load	↑
8.3: Metric k-Center	Greedy: Keep picking the vertex farthest from its current center	↑
8.4: Maximum Cut	Local search: Keep improving the current solution by moving one vertex at a time	1/2

A *α-approximation algorithm* is a polynomial-time algorithm that always returns a solution whose value ALG is within a factor α of the optimal value OPT. In other words, for minimization problems, there exists $\alpha \geq 1$ such that ALG satisfies the following on every instance:

$$\mathsf{OPT} \leq \mathsf{ALG} \leq \alpha \cdot \mathsf{OPT}.$$

The *approximation ratio* of an algorithm is the smallest α such that the algorithm is an α-approximation algorithm. Proving that an α-approximation algorithm is "correct" means showing that the algorithm satisfies $\mathsf{ALG} \leq \alpha \cdot \mathsf{OPT}$ on every instance. (For maximization problems, we flip the inequalities above, so $\alpha \leq 1$ and we want $\mathsf{OPT} \geq \mathsf{ALG} \geq \alpha \cdot \mathsf{OPT}$.)

8.1 Vertex Cover

Problem Statement

The input is an undirected graph G. Our goal is to return a minimum vertex cover of G. Describe a 2-approximation algorithm for this problem and prove that it's correct.

VC-Matching(G, k)**:**

```
S = empty set
while G has an edge e = {u, v}:
    add u, v to S
    remove u, v from G
return S
```

Algorithm: Set $S = \emptyset$. If G has an edge $e = \{u, v\}$, add both endpoints of e to S, and remove u and v from G. (Note that when we remove a vertex from a graph, we also remove all edges incident to the vertex.) Repeat this process until G has no edges and return S.

Correctness: Notice that the edges considered by the algorithm form a matching M, so $|\mathsf{ALG}| = 2 \cdot$

$|M|$. Furthermore, each vertex in OPT can cover at most one edge in M, so $|\mathsf{OPT}| \geq |M|$. Thus, $|\mathsf{ALG}| = 2 \cdot |M| \leq 2 \cdot |\mathsf{OPT}|$.

8.2 Load Balancing

Scheduling is a rich topic with many applications. In a typical scheduling problem, there are n jobs and m machines, and we need to assign each job to a machine. Every job has a length, and the *load* on a machine is the sum of the lengths of the jobs assigned to that machine. The *makespan* of an assignment is the maximum load created by that assignment.

Problem Statement

The input is (ℓ, m), where ℓ is an array of n positive integers representing job lengths and m is the number of machines. Our goal is to return an assignment of jobs to machines that minimizes the makespan. Describe a 2-approximation algorithm for this problem and prove that it's correct.

Algorithm: The algorithm is greedy: iterate through the jobs (in any order), assigning each one to the machine with the smallest load so far.

Remark

Notice that this algorithm makes its final decisions without knowing what the future holds. Algorithms that satisfy this property are known as *online* algorithms.

Correctness: Suppose ALG achieves a makespan of T, the load of $i \in [m]$ is T, and k was the last job assigned to i. When ALG assigned k to i, i had the smallest load. This means the load on every machine is at least $T - \ell[k]$, so $\sum_{j=1}^{n} \ell[j] \geq m(T - \ell[k])$. Rearranging yields

$$T \leq \frac{\sum_{j=1}^{n} \ell[j]}{m} + \ell[k].$$

Now let T^* denote the optimal makespan. Since the average load in any assignment is $\sum_j \ell[j]/m$, we have $T^* \geq \sum_j \ell[j]/m$. Combining this with the inequality above, we get

$$T \leq \frac{\sum_{j=1}^{n} \ell[j]}{m} + \ell[k] \leq T^* + T^* = 2T^*,$$

where $\ell[k] \leq T^*$ because OPT must assign job k to some machine.

An improved analysis

In ALG, since every machine has load at least $T - \ell[k]$ when k was assigned to i, the sum of all job lengths *excluding job* i is at least $m(T - \ell[k])$. Thus, we can improve the analysis from above:

$$T \leq \frac{\sum_{j=1}^{n} \ell[j] - \ell[k]}{m} + \ell[k] = \frac{\sum_{j=1}^{n} \ell[j]}{m} + \left(1 - \frac{1}{m}\right) \ell[k] \leq \left(2 - \frac{1}{m}\right) T^*.$$

This implies that the algorithm is a $(2 - 1/m)$-approximation algorithm.